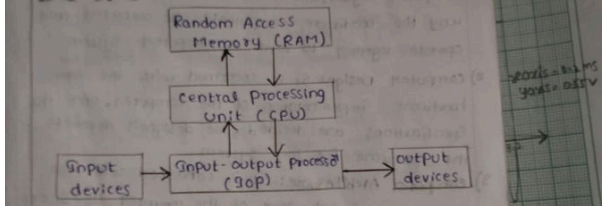
Digital Computer:

A Digital computer is an Electronic computing device that performs arithmetic and logical operations using the binary system "0's & 1's" It accepts data as input, stores the data and gives the required output accurately and quick

Block diagram:-



It consists of three parts

1) CPU (central Processing unit)
2) RAM (Random Access Memory)
3) IOP (Input-output Processor)

1) **CPU:-** It is the Brain of the computer. It consists of ALU, a set of registers and control circuits for fetching and executing instructions

2) **RAM:-** The computer consists of memory to store instructions and data. this memory is called Random Acess memory

3) **I/P- o/p Processor:(IOP):-** The IOP contains Electric circuits for communicating and controlling the transfer of information b/w the computer and outside the world

I/P & o/p devices are keyboard, mouse, joystick, Scanner, printer etc

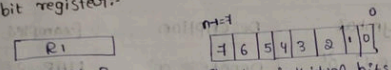
Definations

Register Transfer Language

The Symbolic notation that used to describe the micro-operation transfers among registers is called Register Transfer Language

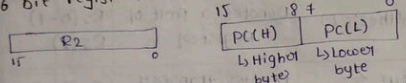
Registers:- A register is a group of flip-flops that stores binary information and each flip-flop represents one bit

8 bit register:-



Register R — Showing individual bits

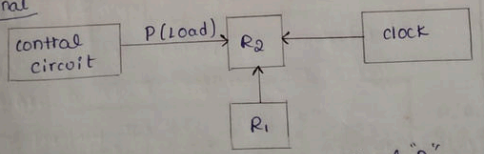
16 bit register:-



Divided into Two parts

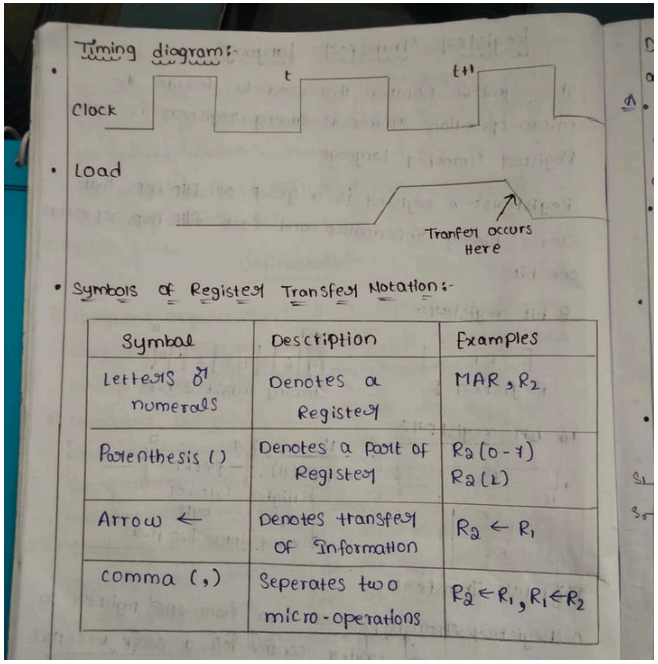
Register Transfer:-

A Register Transfer moves data from one register to another register, under control of a clock & control signal



Control circuit → generates the control signal "P"
R_1 → Source Register
R_2 → destination Register
clock → Synchronizes the transfer

Timing diagram:-

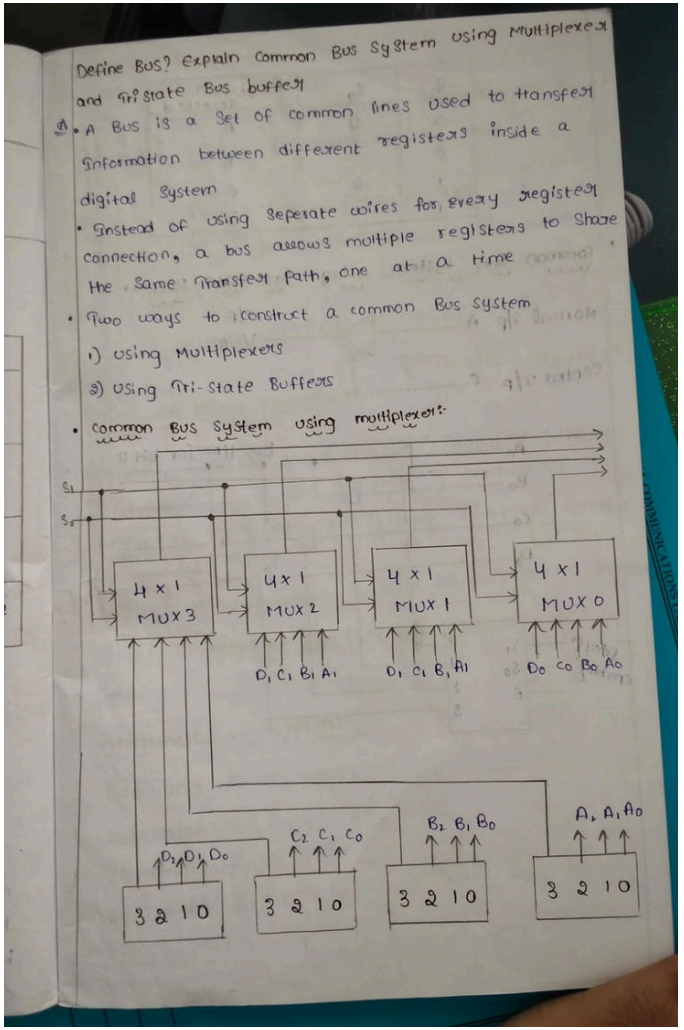


- Symbols of Register Transfer Notation:-

Symbol	Description	Examples
Letters & numerals	Denotes a Register	MAR, R_2
Parenthesis ( )	Denotes a part of Register	$R_2(0-7)$ $R_2(L)$
Arrow ←	Denotes transfer of Information	$R_2 \leftarrow R_1$
comma (,)	Seperates two micro-operations	$R_2 \leftarrow R_1, R_1 \leftarrow R_2$

Define Bus? Explain common Bus System using Multiplexers and Tristate Bus buffers

- A Bus is a set of common lines used to transfer information between different registers inside a digital system
- Instead of using seperate wires for every register connection, a bus allows multiple registers to share the same transfer path, one at a time
- Two ways to construct a common Bus system
  1) using Multiplexers
  2) Using Tri-state Buffers
- common Bus System using multiplexers:

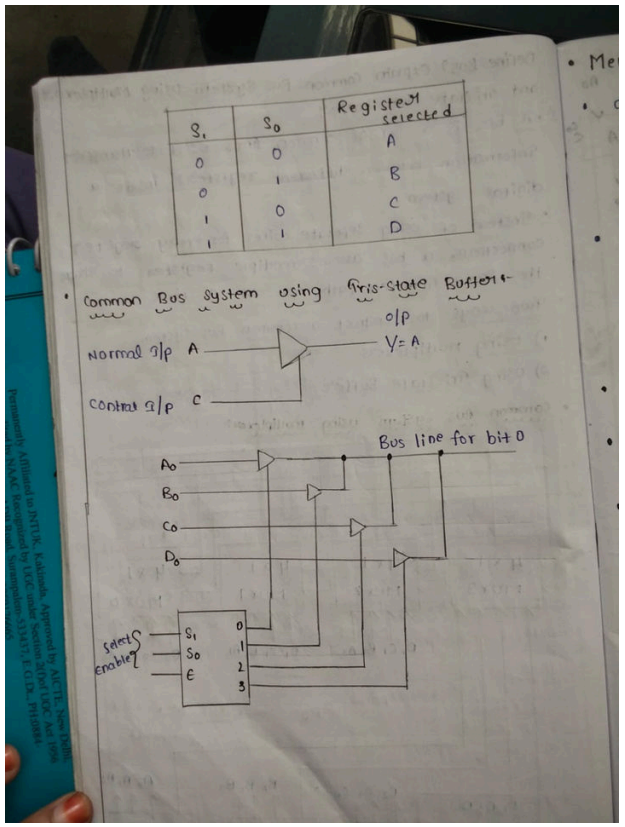


Common Bus System using Multiplexer – Working

1. **Multiplexers** select the data from one of the registers based on the **selection lines (S1, S0)**.
2. The selected register's data is placed on the **common bus**.
3. This common bus transfers the data to the **required destination register**.
4. Thus, **one register can send data to another register through the common bus** using multiplexers.

State Bus Buffers:

A bus system can be constructed with three-state gates instead of multiplexers.
A three-state gate is a digital circuit that exhibits three states.
Two of the states are signals equivalent to logic 1 and 0 as in a conventional gate.
The third state is a *high-impedance state*.
The high-impedance state behaves like an open circuit, which means that the output is disconnected and does not have logic significance.
Because of this feature, a large number of three-state gate outputs can be connected with wire to form a common bus line without endangering loading effects.



Arithmetic micro operations

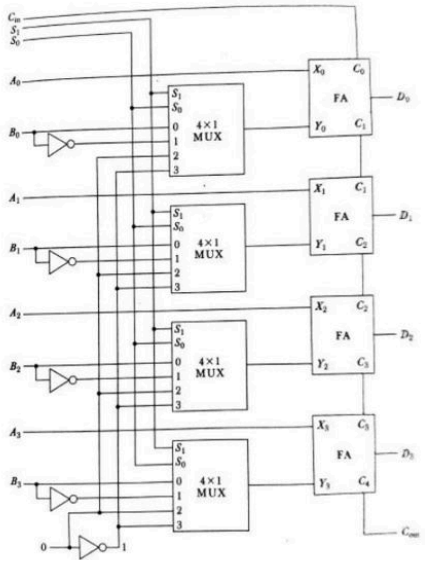
Arithmetic Micro-operations:

- The basic arithmetic micro-operations are
  - Addition
  - Subtraction
  - Increment
  - Decrement
  - Shift
- The arithmetic Micro-operation defined by the statement below specifies the add micro-operation.

R3 ← R1 + R2

Arithmetic Circuit:

- The basic component of an arithmetic circuit is the parallel adder.
- By controlling the data inputs to the adder, it is possible to obtain different types of arithmetic operations.
- The diagram of a 4-bit arithmetic circuit is shown in Fig. 4-9. It has four full-adder circuits that constitute the 4-bit adder and four multiplexers for choosing different operations.



Select S_1	Select S_0	C_{in}	Input Y	Output $D = A + Y + C_{in}$	Microoperation
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with carry
0	1	0	$\bar{B}$	$D = A + \bar{B}$	Subtract with borrow
0	1	1	$\bar{B}$	$D = A + \bar{B} + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A

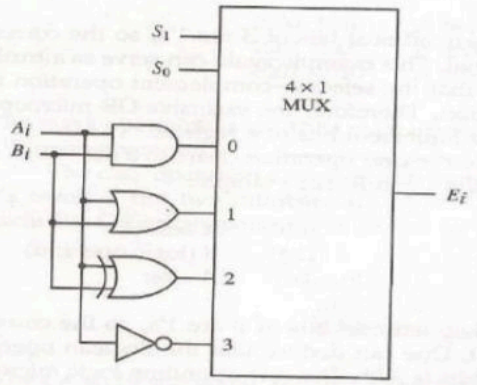
Logic microoperations

: Micro-operations:

- Logic microoperations specify binary operations for strings of bits stored in registers.
- These operations consider each bit of the register separately and treat them as binary variable
- For example, the exclusive-OR microoperation with the contents of two registers RI and R2 is symbolized by the statement

$$P: \quad R1 \leftarrow R1 \oplus R2$$

Figure 4-10 One stage of logic circuit.



(a) Logic diagram

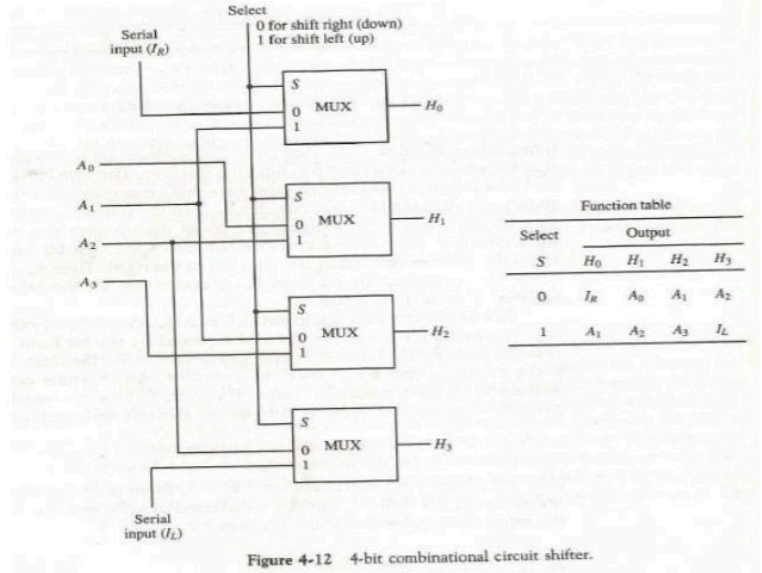
S_1	S_0	Output	Operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \bar{A}$	Complement

(b) Function table

shift microopertaions

ıift Microoperations:

- Shift microoperations are used for serial transfer of data.
- The contents of a register can be shifted to the left or the right.
- During a shift-left operation the serial input transfers a bit into the rightmost position.
- During a shift-right operation the serial input transfers a bit into the leftmost position.
- There are three types of shifts: logical, circular, and arithmetic.

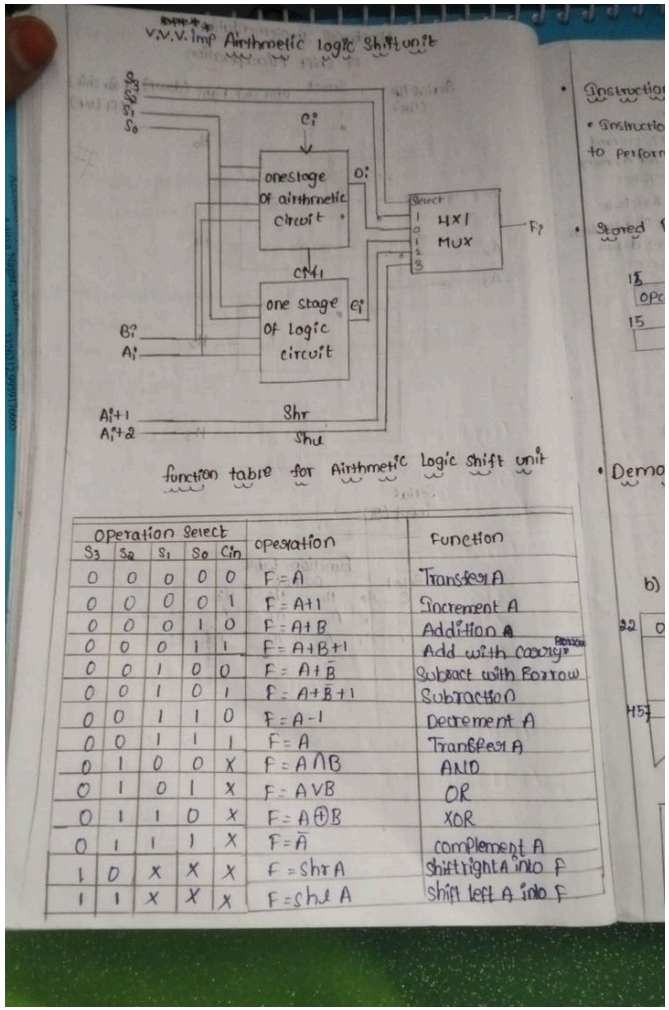


Function table

Select	Output			
S	H_0	H_1	H_2	H_3
0	I_R	A_0	A_1	A_2
1	A_1	A_2	A_3	I_L

Figure 4-12 4-bit combinational circuit shifter.

Arithmetic logic shift unit



1. **ALSU** is a combinational circuit used to perform **arithmetic, logical and shift operations** on binary data.

2. In the given diagram, **A and B are the main data inputs** given to the arithmetic and logic circuits.

3. The **arithmetic circuit** performs operations such as **transfer A, increment A, addition, subtraction and decrement**.

4. The arithmetic circuit also takes **Cin (Carry input)** for performing arithmetic operations with carry.

5. The **one-stage logic circuit** performs logical operations such as **AND, OR, XOR and complement of A**.

6. The outputs of the **arithmetic circuit and logic circuit** are given to the **4 × 1 MUX**.

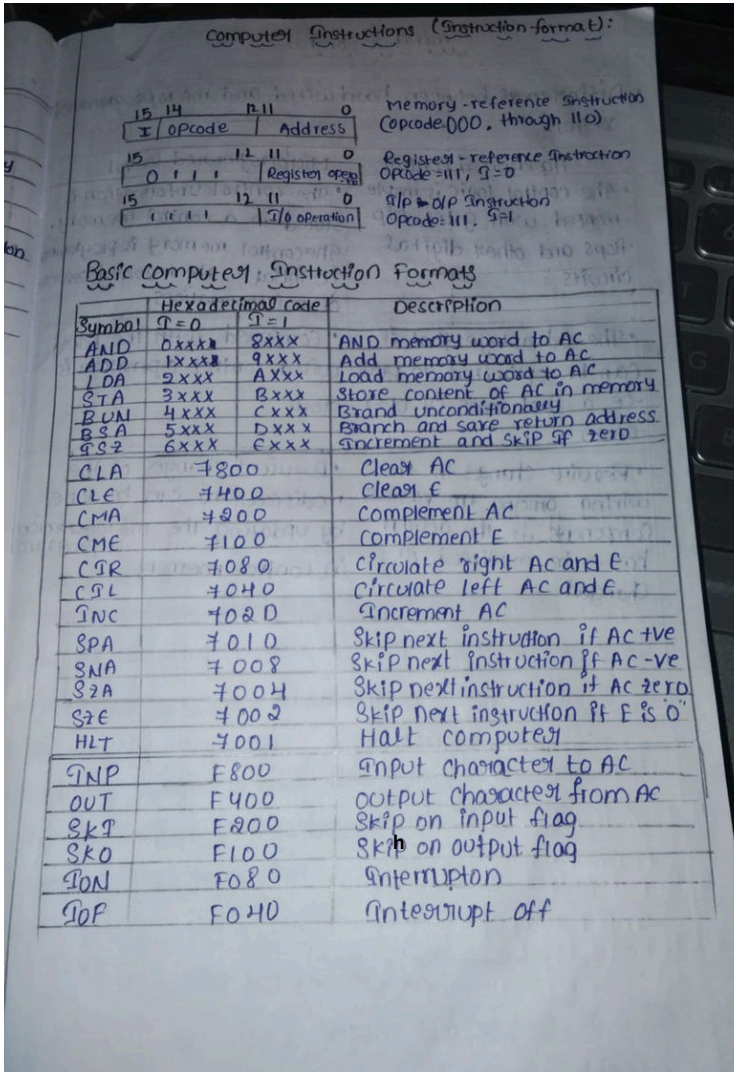
7. The **select lines S_3, S_2, S_1 and S_0** decide which operation has to be performed and which result should be selected.

8. For different combinations of select lines, different operations are obtained at the output **F**.

9. The unit also provides **shift operations**, such as **shift right and shift left**, using the corresponding control/select inputs.

10. Thus, the ALSU can perform several operations using a **single common output F**.

6th ques



Computer Instructions (Instruction format):

Memory-reference instruction (Opcode 000, through 110)

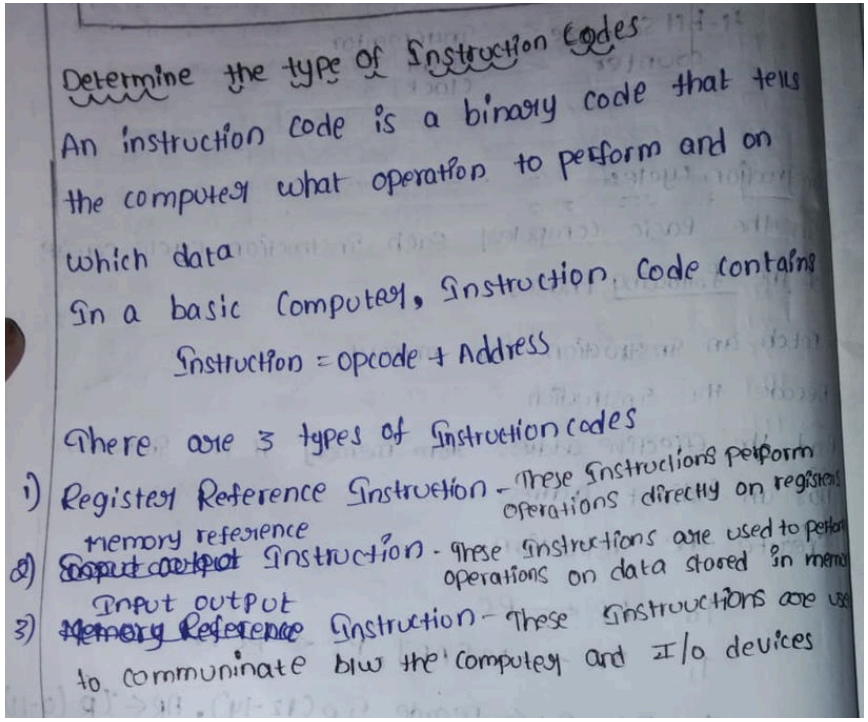
Register-reference instruction Opcode = 111, I = 0

I/p → O/p instruction Opcode = 111, I = 1

Basic Computer Instruction Formats

Symbol	Hexadecimal code I = 0	I = 1	Description
AND	0xxx	8xxx	AND memory word to AC
ADD	1xxx	9xxx	Add memory word to AC
LDA	2xxx	Axxx	Load memory word to AC
STA	3xxx	Bxxx	Store content of AC in memory
BUN	4xxx	Cxxx	Brand unconditionary
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear AC
CLE	7400		Clear E
CMA	7200		Complement AC
CME	7100		Complement E
CIR	7080		Circulate right AC and E
CIL	7040		Circulate left AC and E
INC	7020		Increment AC
SPA	7010		Skip next instruction if AC +ve
SNA	7008		Skip next instruction if AC -ve
SZA	7004		Skip next instruction if AC zero
SZE	7002		Skip next instruction if E is 0
HLT	7001		Halt computer
INP	F800		Input character to AC
OUT	F400		Output character from AC
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt
IOF	F040		Interrupt off

7th ques



Determine the type of Instruction Codes

An instruction code is a binary code that tells the computer what operation to perform and on which data

In a basic Computer, Instruction Code contains

Instruction = opcode + Address

There are 3 types of Instruction codes

1) Register Reference Instruction - These instructions perform operations directly on registers
2) ~~Input output~~ Memory reference Instruction - These instructions are used to perform operations on data stored in memory
3) ~~Memory Reference~~ Input output Instruction - These instructions are used to communicate b/w the computer and I/o devices

Register Reference

$D_7I'T_3 = r$ (common to all register-reference instructions)
$IR(i) = B_i$ [bit in $IR(0\text{–}11)$ that specifies the operation]

	r:	$SC \leftarrow 0$	Clear SC
CLA	rB_{11}:	$AC \leftarrow 0$	Clear AC
CLE	rB_{10}:	$E \leftarrow 0$	Clear E
CMA	rB_9:	$AC \leftarrow \overline{AC}$	Complement AC
CME	rB_8:	$E \leftarrow \overline{E}$	Complement E
CIR	rB_7:	$AC \leftarrow$ shr AC, $AC(15) \leftarrow E$, $E \leftarrow AC(0)$	Circulate right
CIL	rB_6:	$AC \leftarrow$ shl AC, $AC(0) \leftarrow E$, $E \leftarrow AC(15)$	Circulate left
INC	rB_5:	$AC \leftarrow AC + 1$	Increment AC
SPA	rB_4:	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$	Skip if positive
SNA	rB_3:	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$	Skip if negative
SZA	rB_2:	If $(AC = 0)$ then $PC \leftarrow PC + 1$	Skip if AC zero
SZE	rB_1:	If $(E = 0)$ then $(PC \leftarrow PC + 1)$	Skip if E zero
HLT	rB_0:	$S \leftarrow 0$ (S is a start–stop flip-flop)	Halt computer

Memory reference

TABLE 5-4 Memory-Reference Instructions

Symbol	Operation decoder	Symbolic description
AND	D_0	$AC \leftarrow AC \wedge M[AR]$
ADD	D_1	$AC \leftarrow AC + M[AR]$, $E \leftarrow C_{out}$
LDA	D_2	$AC \leftarrow M[AR]$
STA	D_3	$M[AR] \leftarrow AC$
BUN	D_4	$PC \leftarrow AR$
BSA	D_5	$M[AR] \leftarrow PC$, $PC \leftarrow AR + 1$
ISZ	D_6	$M[AR] \leftarrow M[AR] + 1$, If $M[AR] + 1 = 0$ then $PC \leftarrow PC + 1$

Input output

TABLE 5-5 Input-Output Instructions

$D_7IT_3 = p$ (common to all input–output instructions)
$IR(i) = B_i$ [bit in $IR(6\text{–}11)$ that specifies the instruction]

	p:	$SC \leftarrow 0$	Clear SC
INP	pB_{11}:	$AC(0\text{–}7) \leftarrow INPR$, $FGI \leftarrow 0$	Input character
OUT	pB_{10}:	$OUTR \leftarrow AC(0\text{–}7)$, $FGO \leftarrow 0$	Output character
SKI	pB_9:	If $(FGI = 1)$ then $(PC \leftarrow PC + 1)$	Skip on input flag
SKO	pB_8:	If $(FGO = 1)$ then $(PC \leftarrow PC + 1)$	Skip on output flag
ION	pB_7:	$IEN \leftarrow 1$	Interrupt enable on
IOF	pB_6:	$IEN \leftarrow 0$	Interrupt enable off

8th ques

Introduction:

A program stored in the memory consists of a sequence of instructions.

The computer executes the program by performing one instruction cycle for each instruction.

Each instruction cycle is divided into different phases or steps.

1. Fetch the Instruction

The CPU fetches the next instruction from the memory.

The address of the instruction is obtained from the **Program Counter (PC)**.

The fetched instruction is placed in the **Instruction Register (IR)**.

2. Decode the Instruction

The instruction stored in the IR is decoded by the **Control Unit**.
The opcode is identified to determine which operation has to be performed.
The address field and addressing mode are also identified.

3. Read Effective Address

If the instruction uses **indirect addressing**, the effective address is read from memory.
The address obtained from the instruction is used to find the actual address of the operand.
If direct addressing is used, this step is not required.

4. Execute the Instruction

The decoded instruction is finally executed by the CPU.
The required operation such as **ADD, LDA, STA, AND** etc. is performed.
After execution, the CPU goes back to the **fetch phase** to execute the next instruction.